

SQL Problem-Solving Framework

Trainee Version

From remembering SQL syntax to recognizing patterns, decomposing problems, constructing queries, and validating results.

Core workflow

Question → Pattern → Grain → Technique → Layers → SQL → Validation

SQL Problem-Solving Framework

From “remember SQL syntax” to “recognize the problem pattern and construct the solution.”

Core workflow:

Question → Pattern → Grain → Technique → Layers → SQL → Validation

1. SQL Learning Architecture

- Concept Mastery — understand SQL functionality.
- Problem-Solving Framework — learn how to break a business question into pieces.
- Combination Problems — understand how multiple SQL techniques work together.
- Assessment — validate your ability to solve unfamiliar problems independently.

Problem-solving flow:

Output Grain → Tables → Columns → Calculations → Technique → Layers → Validation

2. The Core Problem-Solving Framework

The most important question:

“What does one row in my final output represent?”

That is the output grain.

Example 1

Question: Find total sales for each customer.

One output row = ONE CUSTOMER

Therefore: GROUP BY customer

Example 2

Question: Find monthly sales for each customer.

One output row = ONE CUSTOMER + ONE MONTH

Therefore: GROUP BY customer, month

Example 3

Question: Find the top 3 products in each category.

One output row = ONE PRODUCT within ONE CATEGORY

Ranking is required within category: PARTITION BY category

3. Pattern Dictionary

Question wording / pattern	Think about	Likely technique
Highest salary	Maximum value	MAX()
Lowest salary	Minimum value	MIN()
Average salary	Average	AVG()
Total sales	Aggregation	SUM()
Number of employees	Counting	COUNT()
Customers with more than 5 orders	Aggregate condition	GROUP BY + HAVING
Above average salary	Compare with overall average	Subquery

Question wording / pattern	Think about	Likely technique
Above departmental average	Compare with group average	Correlated subquery / window
Top 3 employees	Global ranking	ORDER BY + LIMIT / window
Top 3 per department	Ranking within group	ROW_NUMBER() / RANK()
Previous transaction	Previous row	LAG()
Next transaction	Next row	LEAD()
Running total	Cumulative calculation	SUM() OVER()
Ranking	Relative position	RANK() / DENSE_RANK()
Remove duplicates	Number rows	ROW_NUMBER()
Latest customer order	Latest row	ROW_NUMBER() / MAX()
First transaction	Earliest row	ROW_NUMBER() / MIN()
Same manager	Compare rows in same table	Self Join
Monthly sales	Group by month	GROUP BY + DATE
Previous month sales	Compare months	LAG()
Month-over-month growth	Current vs previous	LAG() + calculation
Customers who never ordered	Missing relationship	LEFT JOIN / NOT EXISTS
Products never sold	Missing relationship	LEFT JOIN / NOT EXISTS
Above departmental average	Group comparison	Window / subquery
Top N in each category	Partitioned ranking	Window function
Complex multi-stage problem	Multiple transformations	CTE / derived table
Reuse intermediate result	Named intermediate dataset	CTE
Filter aggregated result	Condition after grouping	HAVING

4. SQL Training Ladder

Level	Skills
1	SELECT, WHERE
2	JOIN
3	GROUP BY, aggregates
4	HAVING
5	Subqueries
6	CTE
7	Window functions
8	Date/time
9	Combination problems
10	Real-world SQL problems

5. Practice Database

E-Commerce Sales Database

The schema supports SELECT, filtering, joins, aggregation, subqueries, CTEs, window functions, date/time analysis and combination problems.

Relationship:

customers (1:N) orders (1:N) order_items (N:1) products (N:1) categories

6. Database Setup — MySQL 8+

Categories

```
CREATE TABLE categories (  
    category_id INT PRIMARY KEY,  
    category_name VARCHAR(50)  
);  
  
INSERT INTO categories VALUES  
(1, 'Electronics'),  
(2, 'Furniture'),  
(3, 'Books');
```

Products

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    category_id INT,  
    FOREIGN KEY (category_id)  
        REFERENCES categories(category_id)  
);  
  
INSERT INTO products VALUES  
(101, 'Laptop', 1),  
(102, 'Mouse', 1),  
(103, 'Keyboard', 1),  
(201, 'Chair', 2),  
(202, 'Desk', 2),  
(301, 'SQL Book', 3),  
(302, 'Python Book', 3);
```

Customers

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    city VARCHAR(50)  
);  
  
INSERT INTO customers VALUES  
(1, 'Arun', 'Chennai'),  
(2, 'Priya', 'Bangalore'),  
(3, 'Rahul', 'Mumbai'),  
(4, 'Sneha', 'Chennai'),  
(5, 'Kiran', 'Hyderabad');
```

Orders

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE,  
    FOREIGN KEY (customer_id)  
        REFERENCES customers(customer_id)  
);  
  
INSERT INTO orders VALUES  
(1001, 1, '2026-01-05'),  
(1002, 2, '2026-01-10'),  
(1003, 1, '2026-02-05'),  
(1004, 3, '2026-02-15'),  
(1005, 2, '2026-03-01'),  
(1006, 4, '2026-03-10'),  
(1007, 1, '2026-03-20'),  
(1008, 3, '2026-04-05'),  
(1009, 5, '2026-04-15');
```

Order Items

```
CREATE TABLE order_items (  
    order_item_id INT PRIMARY KEY,
```

```

    order_id INT,
    product_id INT,
    quantity INT,
    unit_price DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

```

INSERT INTO order_items VALUES
(1, 1001, 101, 1, 80000),
(2, 1001, 102, 2, 1000),
(3, 1002, 201, 2, 5000),
(4, 1003, 103, 1, 2500),
(5, 1003, 301, 2, 800),
(6, 1004, 101, 1, 80000),
(7, 1005, 202, 1, 12000),
(8, 1006, 302, 3, 900),
(9, 1007, 102, 1, 1000),
(10, 1008, 201, 1, 5000),
(11, 1008, 301, 1, 800),
(12, 1009, 103, 2, 2500);

```

7. SELECT

Simple

Problem: Display the name and city of every customer.

ONE ROW = ONE CUSTOMER

```

SELECT customer_name, city
FROM customers;

```

Medium

Problem: Display each product with its product name and category ID.

Required table: products. Required columns: product_name, category_id. No join is required because category_id already exists in products.

```

SELECT product_name, category_id
FROM products;

```

8. WHERE

Simple

Problem: Find all customers from Chennai.

Pattern: Find records → Condition → WHERE

```

SELECT *
FROM customers
WHERE city = 'Chennai';

```

Medium

Problem: Find all orders placed after February 1, 2026.

Pattern: Date condition → WHERE → Compare date

```

SELECT *
FROM orders
WHERE order_date > '2026-02-01';

```

9. JOIN

Simple

Problem: Display each order along with the customer name.

Information exists in two tables → JOIN

```

SELECT
    o.order_id,
    c.customer_name,
    o.order_date
FROM orders o

```

```
JOIN customers c
  ON o.customer_id = c.customer_id;
```

Medium

Problem: Display each order with customer name and the products purchased.

Required information is distributed across orders, customers, order_items and products.

```
SELECT
  o.order_id,
  c.customer_name,
  p.product_name,
  oi.quantity
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id;
```

10. GROUP BY

Simple

Problem: Find the number of orders placed by each customer.

Output grain: ONE ROW = ONE CUSTOMER. Pattern: “for each customer” → GROUP BY customer

```
SELECT
  customer_id,
  COUNT(*) AS order_count
FROM orders
GROUP BY customer_id;
```

Medium

Problem: Find total quantity of products purchased by each customer.

Pattern: For each customer + Total quantity → GROUP BY + SUM()

```
SELECT
  o.customer_id,
  SUM(oi.quantity) AS total_quantity
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY o.customer_id;
```

11. HAVING

Simple

Problem: Find customers who have placed more than 2 orders.

WHERE filters rows; HAVING filters groups.

```
SELECT
  customer_id,
  COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 2;
```

Medium

Problem: Find customers whose total purchase quantity is greater than 3.

Pattern: JOIN → GROUP BY → SUM → Filter aggregate → HAVING

```
SELECT
  o.customer_id,
  SUM(oi.quantity) AS total_quantity
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY o.customer_id
HAVING SUM(oi.quantity) > 3;
```

12. SUBQUERY

Simple

Problem: Find products whose product ID is greater than the average product ID.

Pattern: Compare value with calculated value → SUBQUERY

```
SELECT *
FROM products
WHERE product_id >
(
    SELECT AVG(product_id)
    FROM products
);
```

Medium

Problem: Find customers who placed more orders than the average number of orders placed by customers.

Layer 1: orders per customer. Layer 2: average order count. Layer 3: compare each customer's count.

```
SELECT
    customer_id,
    COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) >
(
    SELECT AVG(order_count)
    FROM
    (
        SELECT
            customer_id,
            COUNT(*) AS order_count
        FROM orders
        GROUP BY customer_id
    ) x
);
```

13. CTE

Simple

Problem: Create a temporary result containing the number of orders per customer and display it.

CTE = named intermediate dataset.

```
WITH customer_orders AS
(
    SELECT
        customer_id,
        COUNT(*) AS order_count
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM customer_orders;
```

Medium

Problem: Find customers whose order count is greater than the average customer order count.

Subquery → nested logic. CTE → named layers → better readability.

```
WITH customer_orders AS
(
    SELECT
        customer_id,
        COUNT(*) AS order_count
    FROM orders
    GROUP BY customer_id
)
SELECT
    customer_id,
    order_count
FROM customer_orders
WHERE order_count >
(
```

```

SELECT AVG(order_count)
FROM customer_orders
);

```

14. WINDOW FUNCTIONS

Simple

Problem: Display each customer and their order count along with the overall average order count.

GROUP BY collapses rows to one row per group. Window functions keep the rows and add a calculation.

```

WITH customer_orders AS
(
    SELECT
        customer_id,
        COUNT(*) AS order_count
    FROM orders
    GROUP BY customer_id
)
SELECT
    customer_id,
    order_count,
    AVG(order_count) OVER () AS avg_order_count
FROM customer_orders;

```

Medium

Problem: Rank customers based on their total purchase amount.

Pattern: Total sales → GROUP BY customer → Ranking → RANK()

```

WITH customer_sales AS
(
    SELECT
        o.customer_id,
        SUM(oi.quantity * oi.unit_price) AS total_sales
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.customer_id
)
SELECT
    customer_id,
    total_sales,
    RANK() OVER (
        ORDER BY total_sales DESC
    ) AS sales_rank
FROM customer_sales;

```

15. DATE/TIME

Simple

Problem: Display each order and the month in which it was placed.

Extract month from order_date.

```

SELECT
    order_id,
    order_date,
    MONTH(order_date) AS order_month
FROM orders;

```

Medium

Problem: Find total sales for each month.

Required tables: orders + order_items. Revenue = quantity × unit_price. Month = MONTH(order_date).

```

SELECT
    YEAR(o.order_date) AS order_year,
    MONTH(o.order_date) AS order_month,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY
    YEAR(o.order_date),

```

```

    MONTH(o.order_date)
ORDER BY
    order_year,
    order_month;

```

16. Combination Problems

Combination 1 — JOIN + GROUP BY

Find the total sales generated by each customer.

Pattern: Orders + Order Items → JOIN → Revenue → GROUP BY customer

```

SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY o.customer_id;

```

Combination 2 — JOIN + GROUP BY + HAVING

Find customers whose total sales exceed ■10,000.

```

SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY o.customer_id
HAVING SUM(oi.quantity * oi.unit_price) > 10000;

```

Pattern: JOIN → Calculate revenue → GROUP BY customer → HAVING

Combination 3 — JOIN + SUBQUERY

Find customers whose total spending is above the average customer spending.

Layer 1: Calculate customer revenue.

```

SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY o.customer_id;

```

Layer 2: Calculate average customer revenue.

```

SELECT AVG(total_sales)
FROM
(
    SELECT
        o.customer_id,
        SUM(oi.quantity * oi.unit_price) AS total_sales
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.customer_id
) x;

```

Final: Compare each customer total with the average customer total.

```

SELECT
    o.customer_id,
    SUM(oi.quantity * oi.unit_price) AS total_sales
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY o.customer_id
HAVING SUM(oi.quantity * oi.unit_price) >
(
    SELECT AVG(total_sales)
    FROM
    (
        SELECT
            o2.customer_id,
            SUM(oi2.quantity * oi2.unit_price) AS total_sales

```

```

        FROM orders o2
        JOIN order_items oi2
            ON o2.order_id = oi2.order_id
        GROUP BY o2.customer_id
    ) x
);

```

17. Problem Decomposition — Top 2 Products by Revenue per Category

Problem: Find the top 2 products by revenue for every category.

- What is being asked? → Top 2 products.
- Top 2 based on what? → Revenue.
- For what? → Every category.
- What does one output row represent? → ONE PRODUCT within ONE CATEGORY.
- Required tables: products and order_items. Category is available through products.category_id.
- Revenue = quantity × unit_price.
- Product-level revenue requires GROUP BY product.
- Top N + for every category → window function.
- ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY revenue DESC)

Layer 1 — Product Revenue

```

SELECT
    p.product_id,
    p.product_name,
    p.category_id,
    SUM(oi.quantity * oi.unit_price) AS revenue
FROM products p
JOIN order_items oi
    ON p.product_id = oi.product_id
GROUP BY
    p.product_id,
    p.product_name,
    p.category_id;

```

Layer 2 — Add Ranking

```

WITH product_sales AS
(
    SELECT
        p.product_id,
        p.product_name,
        p.category_id,
        SUM(oi.quantity * oi.unit_price) AS revenue
    FROM products p
    JOIN order_items oi
        ON p.product_id = oi.product_id
    GROUP BY
        p.product_id,
        p.product_name,
        p.category_id
)
SELECT
    product_id,
    product_name,
    category_id,
    revenue,
    ROW_NUMBER() OVER
    (
        PARTITION BY category_id
        ORDER BY revenue DESC
    ) AS product_rank
FROM product_sales;

```

Layer 3 — Filter Top 2

```

WITH product_sales AS
(
    SELECT

```

```

        p.product_id,
        p.product_name,
        p.category_id,
        SUM(oi.quantity * oi.unit_price) AS revenue
FROM products p
JOIN order_items oi
  ON p.product_id = oi.product_id
GROUP BY
  p.product_id,
  p.product_name,
  p.category_id
),
ranked_products AS
(
  SELECT
    product_id,
    product_name,
    category_id,
    revenue,
    ROW_NUMBER() OVER
      (
        PARTITION BY category_id
        ORDER BY revenue DESC
      ) AS product_rank
  FROM product_sales
)
SELECT *
FROM ranked_products
WHERE product_rank <= 2;

```

Pattern to recognize:

“Top 2” → RANKING → “For every category” → PARTITION BY → “Based on revenue” → ORDER BY revenue DESC
 → ROW_NUMBER() → Filter rank <= 2

18. Advanced Problem Pattern

Problem: Find the top 2 products by revenue for every category for each month and compare their revenue with the previous month. Calculate the percentage change from the previous month.

Pattern map:

Top 2 products → Revenue → Every category → Every month → Previous month → Percentage change

Business phrase	SQL requirement
Product revenue	SUM(quantity * unit_price)
Every month	Date transformation
Every category	PARTITION BY category
Top 2	ROW_NUMBER()
Previous month	LAG()
Percentage change	Arithmetic calculation
Multiple stages	CTEs

Solution plan:

STEP 1 — Calculate product-month revenue

STEP 2 — Rank products within category + month

STEP 3 — Keep top 2

STEP 4 — Use LAG() to get previous month revenue

STEP 5 — Calculate percentage change

STEP 6 — Validate

19. Universal Question → Pattern → SQL Map

If the question says...	Ask yourself...	Likely solution
each customer	What is my group?	GROUP BY
total	What am I adding?	SUM()
average	What am I averaging?	AVG()
number of	What am I counting?	COUNT()
more than X	Am I filtering groups?	HAVING
above average	What am I comparing against?	Subquery / CTE
highest	Do I need maximum?	MAX() / ranking
top N	Global or within group?	LIMIT / window
top N per group	Ranking within group?	Window
previous	Previous row?	LAG()
next	Next row?	LEAD()
running	Cumulative?	SUM() OVER()
rank	Relative position?	RANK()
dense rank	Ranking without gaps?	DENSE_RANK()
latest	Most recent row?	ROW_NUMBER()
first	Earliest row?	ROW_NUMBER()
same	Compare rows?	Self Join
never	Missing relationship?	LEFT JOIN / NOT EXISTS
monthly	Date grouping?	Date functions
month-over-month	Previous period?	LAG()
percentage change	Current vs previous	Arithmetic + LAG()
multiple stages	Intermediate datasets	CTE

20. Output Grain Exercises

Question A: Find total sales for each customer.

Expected grain: ONE ROW = ONE CUSTOMER

Question B: Find total sales for each customer for each month.

Expected grain: ONE ROW = ONE CUSTOMER + ONE MONTH

Question C: Find the top 3 products in each category.

Expected grain: ONE ROW = ONE PRODUCT + ONE CATEGORY

Question D: Find the monthly revenue and previous month's revenue for every product.

Expected grain: ONE ROW = ONE PRODUCT + ONE MONTH

Question E: Find the top 2 products in every category for every month.

Expected grain: ONE ROW = ONE PRODUCT + ONE CATEGORY + ONE MONTH

21. GROUP BY vs PARTITION BY

GROUP BY

10 rows → GROUP BY department → 3 rows. The original rows are collapsed.

Example:

Department | Total

IT | 500

HR | 300

Sales | 700

WINDOW FUNCTION

10 rows → PARTITION BY department → 10 rows remain + calculation added.

Example:

Employee | Dept | Salary | Dept Avg

A | IT | 100 | 150

B | IT | 200 | 150

C | HR | 300 | 350

D | HR | 400 | 350

22. Assessment Structure

- Easy — recognize one pattern: WHERE, JOIN, GROUP BY.
- Medium — recognize two patterns: JOIN + GROUP BY; GROUP BY + HAVING; JOIN + SUBQUERY.
- Hard — recognize three or more patterns: JOIN + GROUP BY + HAVING + SUBQUERY.
- Milestone — discover the pattern independently: Business Question → Identify grain → Identify pattern → Build layers → SQL.

Suggested module practice: 10 Easy + 10 Medium + 5 Hard. Subquery/CTE and Window Function modules also include 5 Milestone problems.

23. Weekly Learning Structure

Week 1 — Foundations

- SELECT, WHERE, ORDER BY, DISTINCT, LIMIT

Week 2 — JOIN

- INNER JOIN, LEFT JOIN, Multiple JOINs, Self JOIN

Week 3 — Aggregation

- COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING

Week 4 — Subquery + CTE

- Scalar subquery, IN, EXISTS, Correlated subquery, Derived table, CTE

Week 5 — Window Functions

- ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM OVER, AVG OVER, PARTITION BY

Week 6 — Date/Time

- YEAR, MONTH, DATE, DATEDIFF, DATE_ADD, DATE_SUB, Monthly aggregation, Period comparison

Week 7 — Combination

- JOIN + GROUP BY; JOIN + GROUP BY + HAVING; JOIN + SUBQUERY; JOIN + CTE; JOIN + WINDOW

Week 8 — Real-World SQL

- Customer retention, Product performance, Monthly revenue, Top products, Repeat customers, Inactive customers, Customer ranking, Month-over-month growth, Latest transaction, Duplicate detection

24. SQL Problem-Solving Worksheet

PROBLEM:

1. WHAT IS THE QUESTION ASKING?

2. WHAT DOES ONE OUTPUT ROW REPRESENT?

OUTPUT GRAIN: _____

3. WHICH TABLES ARE REQUIRED?

TABLE 1: _____ TABLE 2: _____ TABLE 3: _____

4. WHICH COLUMNS ARE REQUIRED?

5. WHAT CALCULATIONS ARE REQUIRED?

6. WHAT PATTERN DO YOU RECOGNIZE?

- Filtering
- Joining
- Aggregation
- Group filtering
- Subquery
- CTE
- Ranking
- Previous/Next row
- Running total
- Date comparison
- Other: _____

7. BREAK THE PROBLEM INTO LAYERS

Layer 1: _____

Layer 2: _____

Layer 3: _____

8. WRITE THE SQL

9. VALIDATE INTERMEDIATE RESULTS

10. VALIDATE FINAL RESULT

25. Three Questions for Independent SQL Problem Solving

Question 1 — What does one row represent?

This determines GROUP BY and PARTITION BY.

Question 2 — What calculation has to happen first?

This determines Layer 1.

Question 3 — What has to happen after that?

This determines Layer 2, Layer 3, CTE, Subquery or Window Function.

26. Learning Progression

- Stage 1 — SQL Syntax: “I know SELECT and JOIN.”
- Stage 2 — SQL Concepts: “I know what GROUP BY does.”
- Stage 3 — Pattern Recognition: “I recognize this as a ranking problem.”
- Stage 4 — Problem Decomposition: “I know what Layer 1 should do.”
- Stage 5 — Combination Problems: “I can combine multiple techniques.”
- Stage 6 — Independent Solving: “I can solve an unfamiliar business problem.”

Ultimate goal:

Look at an unfamiliar business question, identify the output grain, recognize the SQL pattern, break the problem into logical layers, construct the query, and validate the result.

27. Central Problem-Solving Rule

Don't ask: “Which SQL command should I use?”

Ask: “What transformation does the business question require?”

Business Question → Output Grain → Required Transformation → SQL Pattern → Query Layers → SQL → Validation